Engineering Applications of Artificial Intelligence 124 (2023) 106553

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

An enhanced recommender system based on heterogeneous graph link prediction

Yassine Afoudi *, Mohamed Lazaar, Safae Hmaidi

ENSIAS, Mohammed V University in Rabat, Morocco

ARTICLE INFO

Keywords:
Recommender systems
Graph Neural Network
Link prediction
Improved GraphSAGE

ABSTRACT

Graph-based data has gained popularity in various applications, including social networks, recommendation systems, and knowledge graphs. Detecting missing links between nodes is a critical challenge in processing graph-based data, as it directly impacts the performance of these applications. The framework for inductive representation learning on large graphs called GraphSAGE method is commonly used for identifying node embeddings, but it has limitations in terms of sensitivity to hyperparameters and difficulties in processing large graphs and capturing global graph information. To address these issues, we propose an improved GraphSAGE architecture for link prediction. Our approach leverages advanced aggregation functions and diverse neural network architectures. Through extensive experimentation on benchmark datasets, we demonstrate the superiority of our method in terms of link prediction accuracy compared to standard GraphSAGE. Additionally, by integrating clustering techniques, we develop a robust recommender system that outperforms state-of-the-art techniques. Our findings highlight the practical potential of our methodology for recommendation tasks and underline its significance in graph-based data processing.

1. Introduction

The advancement of technology and the widespread availability of online services have led to faster access to vast amounts of information. However, this has also resulted in online data overload, making it increasingly challenging for users to find useful information. To address this issue, various techniques with lower computational requirements have emerged, facilitating easier and faster access to relevant content. Consequently, there has been a growing interest in the development of recommender systems (Roy and Dutta, 2022). Recommender systems serve as information filtering tools that provide users with personalized and relevant content or information, aiming to reduce user effort and time spent searching for relevant information online.

Recommender systems are now used in a variety of applications, including online (Wang et al., 2019a), books, e-learning (Wang and Wu, 2011), tourism (García-Crespo et al., 2011), movies (Wibisono et al., 2023), music (Jin et al., 2020), e-commerce (Zhang et al., 2019a), news (Lops et al., 2019), specialized research resources (Porcel et al., 2009), television shows, and so on. The demand for innovative and high-quality recommender systems has become increasingly important to deliver personalized recommendations across various applications. Despite significant advancements in recommender systems, there is still a need for further improvements to enhance the quality of suggestions across a broader range of applications. Therefore, conducting comprehensive research on the latest studies in recommender systems, with a focus on diverse application domains, is crucial to drive advancements and address the evolving needs of users. This research will contribute to the development of more effective recommender systems that can cater to a wider variety of applications and provide users with improved personalized suggestions (Roy and Dutta, 2022).

Generally, recommender system development is divided into three stages: shallow models (Wang et al., 2015a), neural network (NN)-based models (Wang et al., 2021) as deep learning (Cheng et al., 2016), factorization-machine based neural network (Guo et al., 2017), Neural collaborative filtering (He et al., 2017), and Graph Neural Networks (GNN) based models as Lightgcn (He et al., 2020) and Graph Convolutional Neural Network (GCN) (Ying et al., 2018). GNN has gained increasing interest in a variety of cutting-edge domains in recent years (Wu et al., 2022). By conducting information propagation on graphs, GNN has the inherent ability to create connections between users, items, and related qualities in recommender systems. The target node or edge is then the recipient of several aggregated neighborhood embeddings. This approach uses matching embeddings to describe high-order structured data as well as multi-hop relevance between graph elements (nodes or edges). GNN has reached unprecedented

* Corresponding author.
E-mail addresses: yassine_afoudi@um5.ac.ma (Y. Afoudi), mohamed.lazaar@um5.ac.ma (M. Lazaar), safae_hmaidi@um5.ac.ma (S. Hmaidi).

https://doi.org/10.1016/j.engappai.2023.106553
Received 29 March 2023; Received in revised form 21 May 2023; Accepted 26 May 2023
Available online 14 June 2023
0952-1976/© 2023 Elsevier Ltd. All rights reserved.

Nomenclature

C_{PCA}	Representation of the movie dataset with only 10 PCA features
C_{U_T}	Cluster that represents the target user
MG_{U_T}	Target user movie preferred year group
U_T	Target user
GAT	Graph Attention Networks
GCN	Graph Convolutional Networks
GNN	Graph neural network
$GRNN$	Graph Recurrent Neural Networks
$LightGCN$	Light Graph Convolutional Networks
$NGCF$	Neural Graph Collaborative Filtering
PCA	Principal Component Analysis
$ReLU$	A rectified linear activation function
$Tanh$	The hyperbolic tangent function

performance and has become state-of-the-art models in recommender systems due to its great modeling ability. The following are the specific benefits of GNNs in recommender systems.

The unification of structured data comes first. There are several forms for the data in recommender systems (Aggarwal and Aggarwal, 2016), and the graph is useful for organizing and unifying them, with nodes and edges representing various entities or attributes. In this way, a correctly constructed GNN may generate high-level representations for particular tasks while also capturing correlations and interactions across several data sources. The modeling of high-order connections is the second benefit. Multi-hop entity information can be linked via a number of levels of embedding propagation on graphs, identifying high-order connections and similarities between users and items. The availability of multiple supervisory signals is the last advantage. Using semi-supervised signals, such as assisting in modeling the target behaviors with other behaviors in multi-behavior recommendation, GNN can reduce the issue of data sparsity in recommender systems. Moreover, self-supervised paradigms are advantageous for GNN-based models (Gao et al., 2022).

The following succinctly describes our contributions to this work:

- Build the improved SAGEGraph model while also putting together the heterogeneous graph.
- Using a Clustering approach to features that have been picked using a Feature Selection technique will result in the creation of a cluster for each video.
- Employ a similarity metric in order to produce top-N neighbors on the users who belong to the same active user cluster and the same movie year group.
- Using the freshly modified SAGEGraph model, obtain the link prediction for the top-N neighbors.
- Offer recommendations.

The remainder of this work is structured as follows: Section 2 includes background information and relevant research, while Section 3 describes the content recommendation process of the suggested model. Testing is done in Section 4 and the results are presented to show how beneficial the recommended paradigm is. Section 5 also discusses the study's limitations and ongoing issues.

2. Context and related works

Information filtering systems (Hanani et al., 2001) of the recommender kind make educated predictions about the products, services, and content that individual users would find most appealing by analyzing their prior actions and comparing them to data about their demographics and other interests.

Users can receive personalized recommendations from recommender systems since these systems are based on the users' individual tastes and habits (Shinoda, 2012). This has the potential to boost user satisfaction, which in turn can enhance sales and strengthen customer loyalty. Users can also benefit from their ability to find new goods that they might not have found on their own otherwise.

One of the biggest challenges to be addressed while creating recommender systems is the "cold start" problem (Jain et al., 2015). This problem refers to the difficulties of offering suggestions for new users or items that do not have sufficient data currently accessible (Lika et al., 2014). Another difficulty is the "sparsity" problem (Idrissi and Zellou, 2020), which arises when there are numerous users and objects but just a small number of interactions between them. This might make it difficult to find patterns in the data.

A wide array of industries, including e-commerce, social networks, the entertainment industry, and educational institutions, all make use of recommender systems. They can be used in online retail to make product suggestions to customers based on the customers' previous browsing and shopping behavior. They are useful in social networks because they may be used to promote friends or groups to users depending on the activities and interests of those users. In the entertainment industry, they can be used to make recommendations to people based on their tastes for movies, television series, or music. In the field of education, they can be used to tailor recommendations for students' course work to their individual learning styles and levels of development.

The construction of recommender systems makes use of a wide variety of approaches, some of which include matrix factorization (Koren et al., 2009), nearest neighbor algorithms (Bobadilla et al., 2013), and deep learning (Zhang et al., 2019b). Matrix factorization is a common method that divides the user-item interaction matrix into two low-rank matrices, which may then be utilized for the purpose of making suggestions. This method makes use of linear algebra to complete the task. The recommendations made by nearest neighbor algorithms are based on the similarities between the people or the items being considered. Approaches known as "deep learning" (Zhang et al., 2022) employ neural networks in order to discover interesting shapes in the user–item interaction data (Huang et al., 2023).

Many types of recommender systems exist, including collaborative filtering systems, content-based recommender systems, and hybrid recommender systems.

- Collaborative filtering (Ben Schafer et al., 2007) is a popular technique used in recommender systems to provide personalized recommendations to users based on their preferences and the preferences of similar users. It works by identifying patterns and similarities in user behavior and leveraging that information to make predictions about items a user might like.
  In collaborative filtering, there are two main approaches: user-based and item-based filtering. In our earlier work (Afoudi et al., 2019a), we constructed a comparison study based on those two approaches to demonstrate the benefits and drawbacks of each methodology as well as which option provides us with significantly better results.
  User-based collaborative filtering identifies users with similar preferences and recommends items that those similar users have liked. It involves calculating similarity scores between users based on their interactions with items and then recommending items that are highly rated by similar users.
  Item-based collaborative filtering, on the other hand, focuses on identifying similar items based on user preferences. It calculates similarity scores between items and recommends items that are similar to the ones a user has already liked or interacted with.
- Content-based recommender systems (Lops et al., 2011) suggest products based on the features and characteristics of items that a user has previously shown interest in or engaged with. The

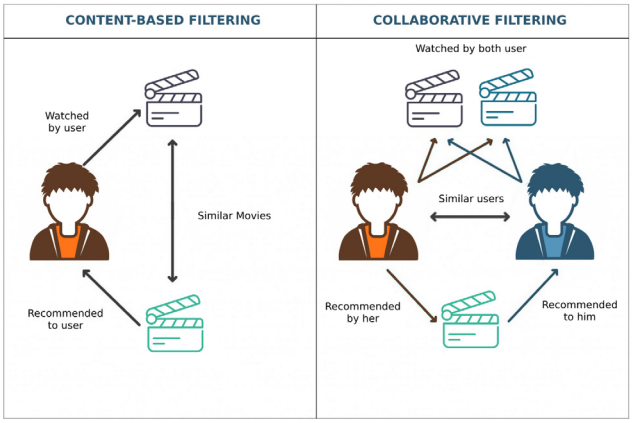

Fig. 1. Collaborative filtering VS content-based filtering.

system analyzes the content or attributes of the items and looks for similarities or patterns among them. By understanding the user's preferences through the content of the items, the system can recommend similar items that align with the user's interests. The main distinction between content-based filtering and collaborative filtering is the source of information used for recommendations. Content-based filtering relies on the properties of the items themselves, such as genre, keywords, or descriptions, to make recommendations. It focuses on understanding the content and using that information to find related items. On the other hand, collaborative filtering, as mentioned earlier, utilizes information from the interactions and preferences of multiple users. It looks for patterns among users' behaviors and recommends items that similar users have shown interest in. Collaborative filtering does not consider the content or attributes of the items explicitly but relies on the collective wisdom of users' preferences.

Fig. 1 illustrates the difference between the two approaches, emphasizing that content-based filtering focuses on item attributes while collaborative filtering leverages user behavior and preferences.

- Hybrid recommender systems (Burke, 2002) aim to leverage the strengths of both content-based and collaborative filtering approaches to provide better and more accurate recommendations. By combining the strategies of content-based and collaborative filtering systems, hybrid systems can overcome the limitations of each individual approach. They can take advantage of the rich information present in item attributes and user behavior simultaneously.

  One common approach in hybrid systems is to use content-based filtering to pre-filter items based on their attributes and then apply collaborative filtering techniques to recommend items to users. This approach combines the personalized nature of collaborative filtering with the ability to capture item characteristics from content-based filtering.

  Hybrid recommender systems have shown to be effective in improving recommendation accuracy and addressing the cold start problem, where new users or items have limited data for accurate recommendations. They can provide more diverse and personalized recommendations by leveraging both user preferences and item attributes.

Graph Neural Networks, also known as GNNs, are becoming increasingly popular as a potential tool for the construction of recommender systems, especially in situations when the data may be represented as a graph. A graph can be used to describe the interactions that take place between users and items in the context of a recommender system.

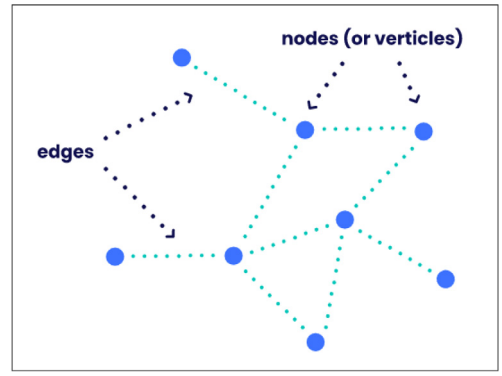

Fig. 2. Data structure in Graph Neural Networks.

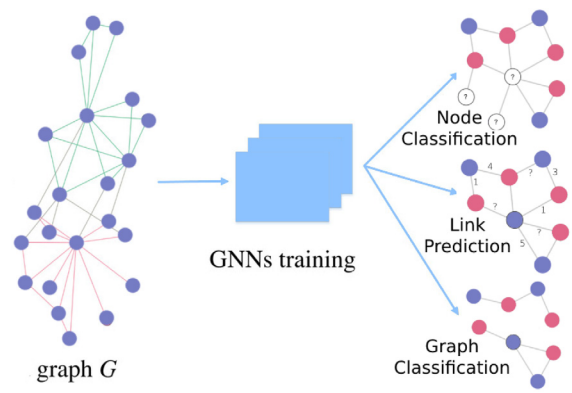

Fig. 3. The main uses of GNN.

2.1. Graph Neural Networks

Graph Neural Networks (Wu et al., 2020), or GNNs (Zhou et al., 2020) for short, are a special kind of neural network (Razmjooy et al., 2019) that can process information in a graph format. Researchers may use GNNs to learn representations of graphs and then perform tasks on them like node classification, link prediction, and graph classification.

In contrast to standard neural networks (Yu et al., 2019), which operate on vector input, GNNs handle data as nodes and edges in a graph. As shown in Fig. 2, a graph is a graphical representation of a network in which each node represents an entity and each edge denotes a connection between nodes.

GNNs are based on the concept of using a message-passing algorithm on a graph to get a desired result. The technique works by having each node update its representation using data it has collected from its neighbors. Until a constant state is achieved, this procedure is iterated upon.

In addition to Graph Recurrent Neural Networks (GRNNs) (Hajiramezanali et al., 2019), Graph Convolutional Networks (GCNs) (Wu et al., 2019), and Graph Attention Networks (GATs) (Veličković et al., 2017) are all types of GNNs. In each variant, the method for gathering neighborhood data and updating node representations is different.

GNNs have been used in a wide range of applications, including computer vision, natural language processing, social network research, and even drug development. They have proven themselves capable of learning state-of-the-art performance across a variety of tasks and acquiring representations of complicated graph-structured data. Fig. 3 illustrates the three main uses of GNN: node classification, link prediction, and graph classification.

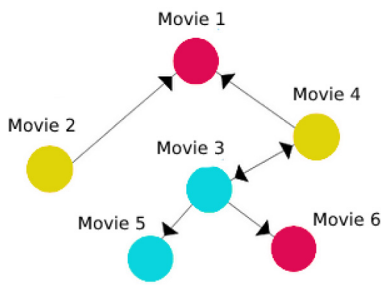

Fig. 4. Graph construction example.

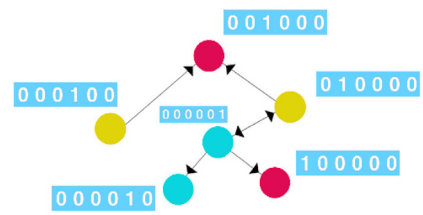

Fig. 5. Example of initialization of node features.

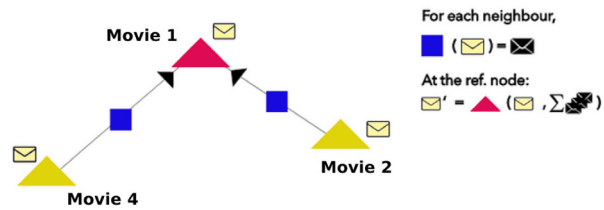

Fig. 6. Message passing process.

2.1.1. Node classification

Common in graph analysis is the process of node classification, which entails assigning a predicted class label to each node in a graph. Classifying nodes in GNNs involves teaching a model to infer each node's class label from its features and the features of its neighbors.

The following are the standard procedures for node classification in GNNs:

- **Graph Construction:** Nodes represent entities and edges indicate links or connections between items in the input data (see Fig. 4).
- **Initialization of Node Features:** Each node in the graph is initially given a feature vector (Example shown in Fig. 5). These characteristics can be created by manually or picked up using a neural network.
- **Message Passing:** GNN model iteratively transmits messages between nodes to update their feature vectors in accordance with the nodes around them (see Fig. 6). In this step, a new feature vector is created by combining the feature vectors of the present node and nearby nodes.
- **Readout:** Using the final feature vectors for each node, the class labels of each node in the graph are predicted. This method of processing feature vectors involves first passing them via a fully connected layer, then a softmax activation function.
- **Training:** The model is trained on a labeled dataset, and certain nodes in the graph have known ground-truth class labels. The loss function for training might be cross-entropy loss or any other suitable loss function for multi-class classification.
- **Inference:** Following training, the model may be used to forecast the network's product nodes' class labels.

Node classification in GNNs has been effectively used in a variety of fields, such as social network research, biology, and recommendation systems. It has demonstrated effectiveness in learning representations of graph-structured data and obtaining cutting-edge performance in a number of tasks.

2.1.2. Link prediction

Identifying whether or not there should be a link (edge) between two nodes in a network is a typical job in graph analysis. When using GNNs, link prediction is done by training a model to determine if a connection exists between any two nodes based on the properties of the nodes themselves and those of their neighbors.

The following stages make up the general strategy for link prediction in GNNs:

- **Graph Construction:** The input data is represented as a graph structure.
- **Initialization of Node Features.**
- **Message Passing:** By transfers messages between nodes to update feature vectors depending on nearby nodes.
- **Link Prediction:** The final feature vectors of each pair of nodes are used to measure the connectivity between them. This may be accomplished using a fully connected layer, a sigmoid activation function, and the dot product or concatenation of the feature vectors.
- **Training:** A labeled dataset containing ground-truth labels for some network node pairs is used to train the model. Binary classification loss functions like binary cross-entropy can be utilized for training.
- **Inference:** After training, the model can predict whether new network nodes are linked.

2.1.3. Graph classification

Graph classification is a problem in graph analysis that entails categorizing the complete graph as opposed to just certain nodes or edges. By building a model to forecast the class label of a graph based on its characteristics and the features of its nodes and edges, graph classification in GNNs is accomplished.

The following stages are part of the general strategy for graph classification in GNNs:

- **Graph Construction:** Each graph represents entities and their relationships or connections as nodes and edges.
- **Initial feature vectors**: For graph nodes and edges Neural networks or handmade features can be used.
- **Message Passing:** This stage combines the feature vectors of surrounding nodes and edges with the current node or edge's to create a new feature vector.
- **Readout:** A fixed-size representation of each graph is created using the final feature vectors for all nodes and edges. Pooling or attention can aggregate node and edge feature vectors.
- **Graph classification:** The class label of each graph is predicted using its fixed-size representation. To do this, the fixed-size representation can be put via a fully linked layer before being activated by a softmax function.
- **Training:** A labeled dataset with ground-truth class labels for each graph trains the model. Training uses cross-entropy loss or another multi-class classification loss function. After training, the model can predict graph class labels.

The main goal of this work is to create a strong recommender system. So, we shall use and settle on the link prediction task.

2.2. Graph construction

First, we will create a graph using the unified formula, which is denoted by the notation G = (V, E). In this notation, V and E refer to

the set of graph nodes and edges, respectively, and each edge in E can link any number of nodes in V. To begin, we will use this notation.

In recent years, GNN-based models have concentrated their efforts on creating specialized networks for the following three categories of graphs:

- A **homogeneous graph** is a sort of graph where there is just one kind of node and edge, and each edge connects just two nodes.
- In a **heterogeneous graph**, Just two nodes are connected by each edge, and both the nodes and the edges can have various forms.
- A **hypergraph**is a particular kind of graph where each edge links more than two nodes.

Many recommender systems, and especially those that rely on graph-based models, include graph construction as a crucial step in the overall process. These techniques use a graph in which the items are nodes and the relationships between them are edges to provide recommendations.

A recommender system's graph may be built in a variety of ways, each appropriate for a distinct use case and set of data. Approaches that are typically used include:

- **User–item graph** (Fan et al., 2019): This approach makes use of a graph with users and items as the two types of nodes. An individual's "interaction history" with a given item is represented by a "edge", which serves to link the person to that item (e.g., viewed, rated, purchased). Collaborative filtering systems, which make suggestions based on the similarities between individuals or items, frequently employ this method.
- **Co-occurrence graph** (Ohsawa et al., 1998): In this method, the graph is made up of nodes for the items and edges for the relationships between the items' co-occurrence or co-purchase. An edge would connect nodes A and B in the graph, for instance, if customers commonly buy goods A and B together. Often, content-based recommender systems employ this strategy.
- **Knowledge graph** (Hogan et al., 2021): In this method, nodes stand in for items and other entities (such as authors, publishers, and genres), while edges signify connections between these entities (e.g., user A bought product B, product B belongs to genre C). Systems for recommending books, movies, and other cultural items frequently employ this strategy.
- **Social network graph** (Myers et al., 2014): In this method, nodes represent users and edges signify the social connections between those individuals (e.g., friend, follower). In social recommender systems, where recommendations are based on the preferences and actions of users' social network connections, this strategy is frequently utilized.

In building and implementing a recommender system, creating a graph is a crucial step. The precise method utilized will depend on the use case in question as well as the data that is available. Knowledge graph serves as the foundation for our line of work.

2.3. Network design

A Graph Neural Network design generally consists of multiple layers of graph convolutional operations that spread information across the graph's nodes. A graph convolutional layer's fundamental principle is to calculate a weighted average of a node's feature vector and the feature vectors of its nearby nodes. A non-linear activation function is then used to this weighted average to create a new feature vector for the node. Throughout the training process, backpropagation is used to learn the weights that will be used in the weighted average.

Depending on the particular application and the properties of the graph data, a GNN's architecture might change. Nonetheless, there are a few standard elements that are frequently present in GNN architectures:

- **Input layer:** This layer is given the graph's initial adjacency matrix and node feature vectors.
- **Graph convolutional layers:** The GNN's primary processing is carried out by the graph convolutional layers, which propagate data across the graph's nodes.
- **Pooling layers:** These layers aggregate information from groups of nodes in the graph. Also, there are many options for aggregating functions, such as mean pooling, LSTM (Fan et al., 2020), max, min, etc. It is essential to build a specific and appropriate one since no one option can perform the best among all recommendation jobs or various datasets. Also, the efficiency of the computation is significantly impacted by the various propagation/aggregation options. As it can be computed quickly, mean pooling, for instance, is frequently employed in GNN-based recommendation models, particularly for graphs containing high-degree nodes, such as extremely popular products.
- **Output layer:** The final output of the GNN is generated by this layer and may take the form of a classification label, a regression value, or a collection of learnt features.

2.4. Recommendation systems with GNN

When there are complex links between items and individuals in the data, GNNs can be an effective tool for developing recommender systems. It is feasible to develop more accurate and meaningful representations of the data and produce more useful suggestions for consumers by utilizing GNNs at various phases of the recommendation process.

2.4.1. Graph Convolutional Networks

GCN (Kipf and Welling, 2016): Iteratively aggregates information from neighbors by approximating the first order eigendecomposition of the graph Laplacian. It specifically updates the embedding by Aggregation as presented in formula (1) and Update function as formula (2):

$$n_v^{(l)} = \sum_{j \in N_v} (d_{vv}^{\frac{-1}{2}} \tilde{a}_{vj} d_{jj}^{\frac{-1}{2}} h_v^{(l)}) \tag{1}$$

$$h_v^{(l+1)} = \delta(W^{(l)} n_v^{(l)}) \tag{2}$$

where $\delta(.)$ is a nonlinear activation function, similar to ReLU, $W^{(l)}$ is the learnable transformation matrix for layer l, $\tilde{a}_{vj}$ is the adjacency weight $\tilde{a}_{vj} = 1$, and $d_{jj} = \sum_k (\tilde{a}_{jk})$ (Wu et al., 2022).

GCNs can be used in recommender systems to model user–item interactions and incorporate additional information about users and items into the recommendation process, such as attributes or textual descriptions.

Building a graph with users and items as nodes and edges representing interactions between them is one way to include GCNs into recommendation systems. Wang et al. (2019) proposed NGCF (Neural Graph Collaborative Filtering) (Wang et al., 2019c), a GCN-based recommendation framework that uses this graph to make personalized recommendations. On several benchmark datasets, they demonstrated that NGCF outperforms traditional recommendation methods such as matrix factorization and deep neural networks.

2.4.2. Graph attention network

Based on the premise that the influence of neighbors is neither constant nor predetermined by the graph structure (Veličković et al., 2017), this method updates the vector of each node by paying attention to its neighbors in order to distinguish between the contributions of neighbors (Myers et al., 2014). It specifically updates the embedding by Aggregation as presented in formula (3) and Update function as formula (4):

$$n_v^{(l)} = \sum_{j \in N_v} \alpha_{vj} h_j^{(l)}, \alpha_{vj} = \frac{e^{ATT(h_v^{(l)} h_j^{(l)})}}{\sum_{k \in N_v} e^{ATT(h_v^{(l)} h_k^{(l)})}} \tag{3}$$

$$h_v^{(l+1)} = \delta(W^{(l)} n_v^{(l)}) \tag{4}$$

where $ATT(.)$ an attention function and a typical is $ATT(.)$ is LeakyReLU $(a^T[W^{(l)}h_v^{(l)} \bigoplus W^{(l)}h_j^{(l)}])$, $W^{(l)}$ is in charge of transforming the node representations during the lth propagation, and a is the learnable parameter (Myers et al., 2014).

The article "Knowledge Graph Attention Network for Recommendation" (Wang et al., 2019b),describes a recommender system called Knowledge Graph Attention Network (KGAT) that uses knowledge graphs and attention mechanisms to generate personalized recommendations for users. The system learns the representations of entities and connections by representing users, items, and their relationships as nodes in a knowledge graph.

2.4.3. Light Graph Convolutional Networks

In the article "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation" (He et al., 2020), LightGCN is a novel graph convolutional neural network (GCN) (Wang et al., 2015b) based recommender system that aims to simplify and improve the performance of existing GCN-based recommendation models. The system addresses the sparsity issue in user–item interaction matrices by using a linear embedding aggregation scheme that only involves user–item interactions. The model removes non-linearities and user-specific biases, simplifying the GCN model.

2.4.4. GraphSage

GraphSage (Hamilton et al., 2017) utilizes a predetermined number of neighbors per node, suggests a mean, sum, maxpooling or aggregator, and relies on a concatenation update operation as shown in Eqs. (5) and (6).

$$n_v^{(l)} = Aggregator_l(\{\ h_u^{(l)}, \forall u \in N_v\ \}) \tag{5}$$

$$h_v^{(l+1)} = \delta(W^{(l)} \cdot [h_v^{(l)} \bigoplus n_v^{(l)}]) \tag{6}$$

GraphSAGE is an inductive learning framework that quickly generates embeddings of unknown vertices based on their known attributes. To construct the embedding vector of a target vertex, the basic idea is to train a function that aggregates the representation of neighboring vertices. Three basic actions make up the process: First, a similar breadth-first traversal is used to sample each vertex in the graph; The second step is the aggregation of information contained in neighbor vertices. Thirdly, a nonlinear activation function is utilized to determine the embedding vector of each vertex in the graph (Song et al., 2022).

A graph neural network model called GraphSAGE may be used to train node embeddings for a variety of graph-based tasks, such as recommender systems. In recommender systems, GraphSAGE can be used to model user–item interactions and make personalized recommendations. One study by Hamilton et al. (2018) demonstrated the effectiveness of GraphSAGE for recommendation systems, achieving state-of-the-art performance on several benchmark datasets.

3. Proposed work

3.1. Improved GraphSAGE

GraphSAGE is a recently developed graph neural network model that has shown promising results in various tasks, including node classification, link prediction, and recommendation. The GraphSAGE model aggregates the feature information of a node's local neighborhood to learn informative embeddings that capture the underlying structure of the graph.

In this work, we propose to investigate the use of GraphSAGE for recommendation tasks and build an improved one.

A framework for inductive representation learning on massive graphs is our improved GraphSAGE. Improved GraphSAGE is used to create low-dimensional vector representations for nodes and is especially beneficial for networks with a rich of node attribute data. This method's major application is in link prediction.

In this section, we outline the GraphSAGE algorithm's suggested improvements for node embedding in graph data. We concentrate on the neighborhood aggregation phase in particular, where we introduce the usage of the Hyperbolic Tangent (Tanh) function as the activation function on the first layer as shown in Eq. (7), followed by Rectified Linear Units (ReLU) on the second layer. Eq. (8) demonstrates that we also restrict the activation of the ReLU function to a maximum size of 6. In addition, during the training phase, we employ Root Mean Square Error (RMSE) as the loss function.

$$Tanh(x) = tanh(x) = \frac{exp(x) - exp(-x)}{exp(x) + exp(-x)} \tag{7}$$

$$ReLU6(x) = min(max(0, x), 6) \tag{8}$$

3.1.1. Neighborhood aggregation with hyperbolic tangent and ReLU

In order to learn node embeddings, the GraphSAGE approach collects the features of a node's immediate neighborhood. The weighted sum of the feature vectors of the neighboring nodes from the initial method is used for this, followed by a nonlinear activation function. We suggest changing this step by activating the first layer using the hyperbolic tangent (Tanh) function, then activating the second layer with ReLU.

The input values are mapped to a range of [−1, 1] via the sigmoidal activation function known as the Tanh function. Specifically if the feature vectors have a wide range of values, this nonlinearity enables the model to capture more complex patterns in the data. A rectified linear activation function, or ReLU function, on the other hand, reduces all negative values to zero. When working with high-dimensional feature vectors, this enables the model to learn sparse representations of the input.

To limit the activation of the ReLU function to a maximum size of 6, we introduce a clipping function that sets any values above 6 to 6. This helps to prevent the model from becoming too sensitive to outliers in the data, which can lead to overfitting.

3.1.2. Loss function

The softmax cross-entropy loss is the loss function utilized in the training phase of the original GraphSAGE algorithm. Root Mean Square Error (RMSE), as an alternative, is what we suggest using as the loss function as mentioned in Eq. (9). The RMSE loss function is frequently used in regression situations in an effort to minimize the disparity between the predicted values and the actual values.

$$L = \sqrt{\frac{1}{N}[\sum(\hat{y} - y)^2]} \tag{9}$$

We intend to improve the model's performance on tasks that require predicting continuous variables, such as node centrality or link prediction, by utilizing RMSE as the loss function. This is because the RMSE loss function is more appropriate for regression problems, as it penalizes large errors more heavily than small errors.

3.2. Proposed architecture

Our proposed approach illustrated in Algorithm 1 and Fig. 7 consists of several steps, including data preprocessing, constructing the heterogeneous graph, developing an Improved SAGEGraph model, applying Spectral Clustering, and generating recommendations using link prediction. The significance of our work lies in its potential to enhance the accuracy and diversity of movie recommendations.

Data Preprocessing: We will use a publicly available movie rating dataset for this work. The dataset contains movie ratings provided by users on a scale of 1–5. We will perform data preprocessing steps such as removing duplicates, handling missing values, and selecting relevant features for our model. We will also apply normalization techniques to ensure that the features are on the same scale.

Heterogeneous Graph Construction: To model the relationships between users and movies, we will construct a heterogeneous graph that

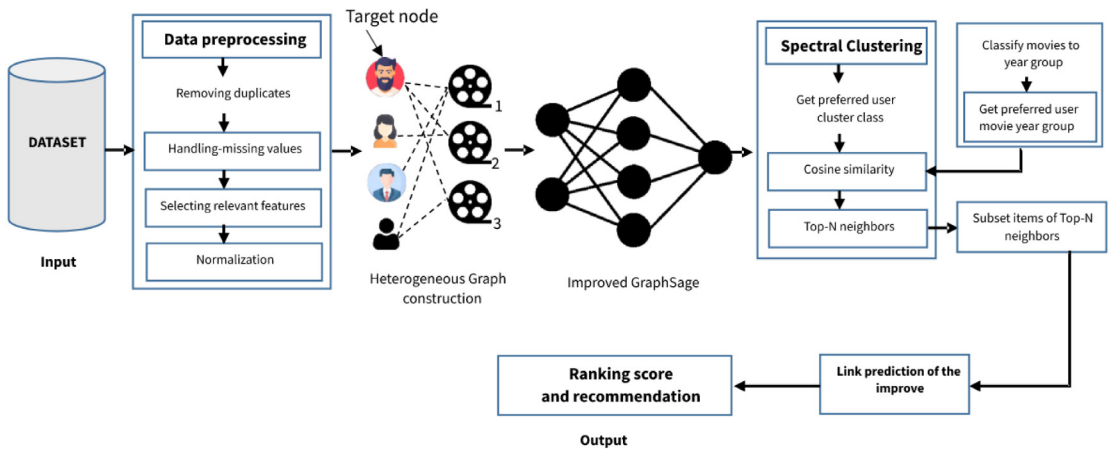

Fig. 7. Proposed architecture.

Algorithm 1 Our Recommender system model

Input: Heterogeneous Graph.
Output: the rating prediction result $\hat{R}$.

1. Build the Improved SAGEGraph model.
2. PCA feature selection. $C_{PCA} = 10$
3. Apply Spectral Clustering on C_{PCA} to create movie clusters $C_i = C_1..C_n$
   $C_i = C_1..C_n \leftarrow SC(C_{PCA})$
4. Determine cluster that represents user U_t
   $C_{U_T} \leftarrow max(C_i, U_T)$
5. Determine user movie year group.
   $MG_{U_T} \leftarrow max(MG_i, U_T)$
6. Apply cosine similarity to generate top-N neighbors
   $MG_{U_T} \leftarrow max(MG_i, U_T)$
7. Get top-N neighbors items $MG_{U_T} \cap C_{U_T}$
8. improved SAGEGraph model for link prediction
   $R' \leftarrow ImprovedSAGEGraph(items)$
9. Arrange the link prediction outcomes and offer a recommendation.
   $\hat{R} = Arrange(R')$

incorporates user and movie nodes as well as their associated features. We will use the torch geometric library to create and analyze the graph. The graph will be represented using adjacency matrices, where each element represents the strength of the relationship between two nodes.

Improved SAGEGraph Model Development: We will develop an Improved SAGEGraph model that utilizes the constructed heterogeneous graph to generate user–item prediction(link prediction). The Improved SAGEGraph model will consist of two layers that perform graph convolutions to learn the node representations.

Spectral Clustering and Active User Identification: To classify movies based on the 10 features obtained by the principal component analysis features reduction approach (based on findings from our earlier work (Afoudi et al., 2019b) on the beneficial effects of feature reduction techniques on recommender systems), we will utilize Spectral Clustering (Luxburg, 2007). We will next calculate the favorite movie year by the same user (Max) and the current user's favorite movie cluster (Max).

Graph theory serves as the basis of the method known as spectral clustering, which is used to identify groups of nodes in a network based on the connections that link them. We may also cluster non-graph data using the method's adaptability.

Cosine Similarity for Neighbor Generation: Based on users in the same active user cluster and the same movie year group, we will utilize the cosine similarity metric below to build the top-N neighbors for the active user.

$$cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}} \tag{10}$$

where A_i and B_i are the ith components of vectors A and B, respectively.

Link Prediction and Recommendation: Based on the Improved SAGEGraph model, we will utilize link prediction to produce recommendations for the subset of items ids of the top-N neighbors. We will use measures like accuracy, recall, and F1-score to assess the model's performance. We will arrange the link prediction results and utilize them to make a suggestion.

4. Experiments

4.1. Datasets for evaluation

4.1.1. MovieLens dataset

Users' ratings from the website Web MovieLens are included in this data collection. The research group GroupLens at the University of Minnesota collected and continues to update this data collection. There are five various versions available: "25m", "latest-small", "100k", "1m" and "20m". In all datasets, information about movies and information about rating are merged under the keyword "movieId".

In our study, we make use of the "latest-small" dataset, which is a small segment of the most recent version of the MovieLens dataset. It is updated and changed over time by GroupLens.

4.1.2. IMDB dataset

We test the effectiveness of our system using the IMDB dataset, which can be accessed and downloaded at https://datasets.imdbws.com/. Every day, the data is updated.

4.2. Implementation

4.2.1. Experimental steps

1. The user interaction dataset (rating dataset) is split into two portions after being imported into our work environment: 75% (75627) serve as the training set and 25% (25209) serve as the test set. Here, we do not even employ the cross-validation method to fix the overfitting problem; instead, we use alternative approaches like the Early Stopping technique and Dropout to deal with the issue. The userId feature of the ratings dataset and movieId on both the ratings and items datasets are then

Table 1
The year group conditions.

Condition	Year group
year < 1900	0
1900 <= year <= 1975	1
1976 <= year <= 1995	2
1996 <= year <= 2003	3
2004 <= year <= 2009	4
2010 <= year	5

preprocessed using the LabelEncoder method. Lastly, we list the titles of each movie and its genre features, giving it a value of 1 if it matches that genre and a value of 0 otherwise.

2. We receive movie cleaned and ordered data after the cleaning and preprocessing phase. Then, using a torch geometric framework, we construct a heterogeneous graph into which we add user node attributes, movie node characteristics, and ratings between users and movies for the purpose of message passing. Finally, using 2 layers and 64 as the dimension of the generated node embeddings for the model's input and output, we construct our improved GraphSAGE framework as described before.
3. Our Improved SAGEGraph architecture is trained using a train graph with 300 iterations, an Adam optimizer with a learning rate parameter of 0.01, and RMSE as the loss function between the link predicted value and the actual value.
4. After the training phase, in order to increase accuracy, we use PCA feature extraction (Banan et al., 2020) to reduce the number of movie features from 20 genres to just 10. We then use Spectral Clustering to create clusters of each movie using the 10 PCA feature selection results, and we assign each user to the class that has received the most positive ratings for the maximum number of cluster movies. Then, we give each movie its production year and classify them into groups using Table 1. Finally, we choose for each user his preferred year group of the films he has seen the most and users were sorted into groups according to their cluster numbers. As a summary, each user is represented by his favorite cluster movie and his preferred year movie group in the output from this step.
5. The final stage is to create our hybrid model and generate suggestions. In general, during this phase, we employ cosine similarity to build the top-5 neighbors on the users who are part of the same active user cluster and similar preferred movie year group. Finally, for the target user, we create a link prediction for each of the top-5 user's items. Following an analysis of every movie the user has viewed, we rate the remaining movies from best to worst and provide a Top-N suggestion to the user.

After constructing our model, we want to check to see if the items that were recommended to a user who was actively using the system during the training phase are on the list of items that the same user saw and highly rated in the test set. This will allow us to test and evaluate the effectiveness of the recommendations.

4.2.2. Experimental platform

Every one of our observations was turned into a Jupyter notebook, and the corresponding code was written in Python. Python is a programming language that comes packaged with a large number of frameworks and libraries that make it simple to write code, which in turn saves a substantial amount of time. Each one of the evaluations was carried out on an HP EliteBook laptop equipped with a 2.4 GHz Intel Core i7 CPU and 16 GB of RAM.

4.2.3. Evaluation

Several methods for providing recommendations have been proposed due to the widespread interest in recommender systems among researchers. Accuracy, robustness, scalability, and so on are only few of the qualities of recommendation systems that might have an impact on the user experience.

It is common practice for a recommendation system to classify the users in a dataset and then present them with a ranked list of suggestions, from best to worst.

In most situations, the user does not care too much about the order of choices; they just want a few good ones. but for practical reasons, it is critical to evaluate the performance of any recommender system.

For the purpose of evaluating a recommendation system's efficacy, in this section we will go through some of the most crucial and widely employed evaluation metrics.

It is common practice to utilize either an offline or online method when evaluating recommendation systems.

The offline Evaluation is the most straightforward and secure method. In general, researchers should divide the available data into training data and test data, and then use the training data to predict the test data.

Online evaluation is performed in real time on the actual users of the system. This evaluation may consist just of evaluating the financial performance before and after the implementation of RS.

In this research, we will employ the offline method based on a database of real user interactions with movies.

MAE: is a metric that measures the average magnitude of absolute differences between the ratings that were predicted and the ratings that were actually given by users. The same scale of rating is used for the Prediction value, and its quality is better when the Mean Absolute Error (MAE) is low. The MAE represented by the following equation (Al-Bakri and Hashim, 2018):

$$MAE = \frac{\sum(i,j)\left|P_{(i,j)} - r_{(i,j)}\right|}{n} \tag{11}$$

n is the total number of users who were tested. The projected rating for user I on item j is represented by $P_{(i,j)}$, whereas the actual rating is represented by $r_{(i,j)}$.

RMSE: is the square root of the sum of the squares of the predicted and actual ratings differences. The RMSE represented by the following equation (Bathla et al., 2021):

$$RMSE = \sqrt{\frac{\sum_{(i,j)}(\left|P_{(i,j)} - r_{(i,j)}\right|)^2}{n}} \tag{12}$$

Precision@k: is the percentage of pertinent top-k set recommendations. The following equation represents the Precision@k. Ramzan et al. (2019):

$$Precision@k = \frac{R}{Tr} \tag{13}$$

Recall@k: is the percentage of pertinent items that appear in the top-k recommendations. The following equation represents the Recall@k (Ramzan et al., 2019):

$$Recall@k = \frac{R}{Nr} \tag{14}$$

Where R stands for the number of recommended items that are relevant at @k, Nr for the total number of items that are relevant, and Tr for the number of items that are recommended at @k.

F-measure: is the Harmonic Mean of Precision and Recall, and it is used to assess the accuracy and efficiency of the suggested recommender system's findings. F-measure represented by the following equation (Ramzan et al., 2019):

$$F - measure@k = 2 \times \frac{(precision \times recall)}{(precision + recall)} \tag{15}$$

where precision=$\frac{TP}{TP+FP}$ and recall=$\frac{TP}{TP+FN}$

With TP = True positive, TN = True negative, FP = False positive and FN = False Negative

MAP and MAR: For each user in the test set, a recommender system generally generates an ordered list of recommendations. Whereas

Table 2
RMSE, MAE and accuracy results for all GNNs models.

Training	Metrics	Models			
		SAGEGraph	Improved SAGEGraph	GATConv	LightGNN
MovieLens (80%)	RMSE	0.8009	**0.6247**	0.8844	0.8233
	MAE	0.6089	**0.3945**	0.6920	0.6291
MovieLens (70%)	RMSE	0.8134	**0.7302**	0.9193	0.8431
	MAE	0.6195	**0.4554**	0.7258	0.6458
IMDB (80%)	RMSE	0.8528	**0.7930**	0.9441	0.8555
	MAE	0.6565	**0.4860**	0.7480	0.6534
IMDB (70%)	RMSE	0.8238	**0.6493**	0.9136	0.8395
	MAE	0.6312	**0.3808**	0.7176	0.6407

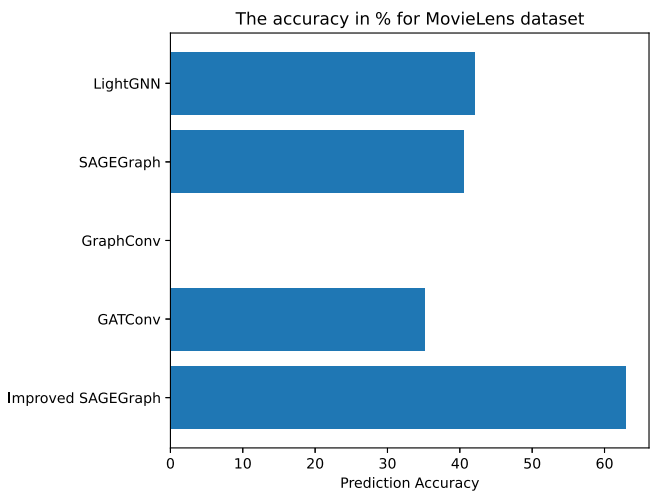

Fig. 8. The accuracy for MovieLens dataset.

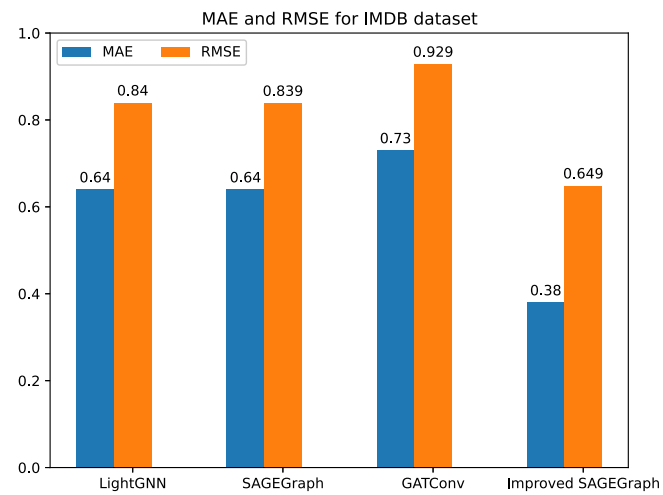

Fig. 9. The MAR and RMSE for IMDB dataset.

MAR@K measures the recommender's recall of every item in the test set that the user gave a favorable rating, MAP@K measures how relevant the list of recommended products is.

Coverage: is the percentage of training data items that the model can successfully recommend on a test set, as shown in the formula below. Covering refers to how well suggestions cover the range of available products and how well suggestions may be made for all possible users. High coverage in a recommender indicates that it has thoroughly explored the products available to its users, and is thus a quality indication.

$$catalogcoverage = \frac{\left|U_{j=1..N} I_L^j\right|}{|I|} \tag{16}$$

Where I_L^j is the set of items contained in the list L returned by the jth recommendations observed during the measurement time. N is used to denote the total number of recommendations observed during the measurement time and I is the set of all available items.

4.2.4. Discussion of the results

Our model, as well as the proposed improved SAGEGraph model, outperformed all other GNN approaches that we evaluated. As shown in Fig. 8, our model achieved an accuracy of 65%, which is significantly higher than the next best model, which achieved an accuracy of 42%. The SAGEGraph model performed even better, achieving an accuracy of 40%.

Our improved SAGEGraph model also demonstrated superior performance in terms of the MAE and RMSE metrics compared to the other GNN approaches. As shown in Table 2, our models achieved smaller MAE and RMSE values, indicating better predictive accuracy compared to the other models.

In addition to demonstrating superior performance on the training dataset, our improved approach also achieved better MAE and RMSE scores on the IMDB dataset compared to the other GNN approaches, as shown in Fig. 9. This indicates that our models are capable of generalizing well to new, unseen data and can make accurate predictions on datasets beyond their training set. Also demonstrated superior time and resource efficiency compared to the other GNN approaches. As shown in Fig. 10, our models required significantly less time and fewer resources to train compared to the other models. The results of our proposed approach demonstrated significant improvements in the Precision, Recall, and F1 measures compared to SAGEGraph or even improved SAGEGraph approaches, as shown in Fig. 11.

Moreover, as shown in Fig. 12, our proposed approach also provides significantly greater coverage than other GNN approaches, indicating that our model is capable of generating recommendations for a larger number of items. This is a critical consideration for recommendation systems where the goal is to provide users with diverse and personalized recommendations. In terms of Mean Average Recall (MAR), our improved SAGEGraph technique and the suggested recommender system achieved better MAR scores than other GNN approaches or popular recommender system, as demonstrated in Figs. 13 and 14. This indicates that our models are capable of making accurate recommendations and achieving high user satisfaction rates.

Additionally, as shown in Fig. 15, our approach is capable of generating high-quality recommendations in a short period of time, making it suitable for real-time recommendation systems where efficiency is a critical consideration.

Overall, the results of this study demonstrate that our proposed approach and the improved SAGEGraph technique are effective and efficient tools for generating accurate and diverse recommendations. These models have the potential to be applied in various real-world applications, including e-commerce, social networks, and content-based platforms, where personalized and efficient recommendations are critical for user satisfaction and engagement.

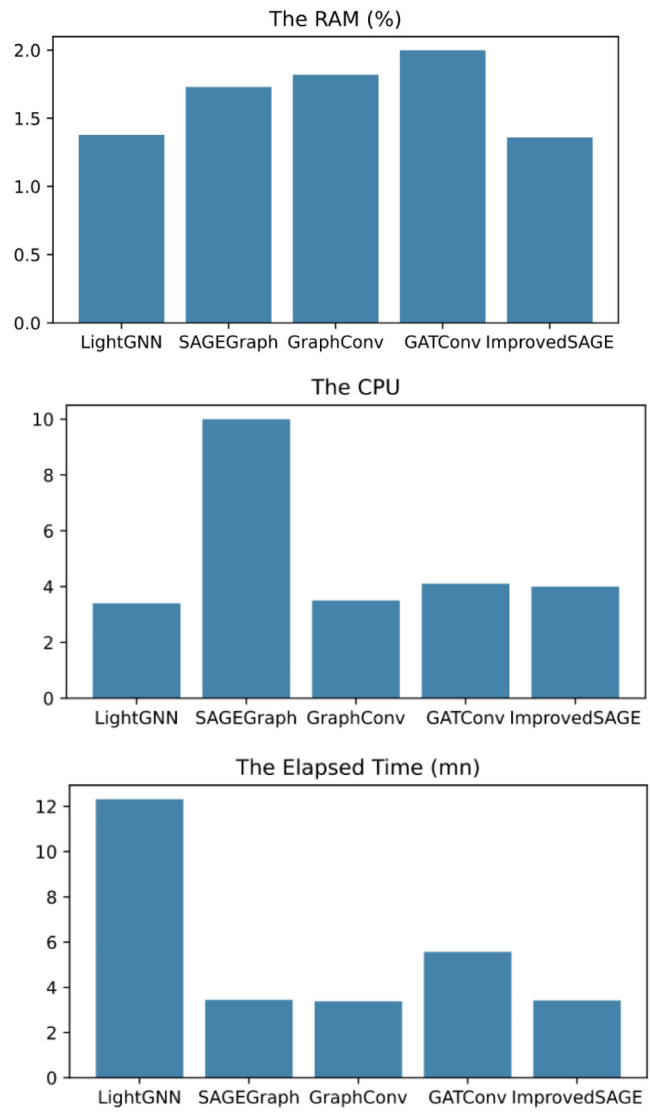

Fig. 10. The measures of resources utilized to train the models.

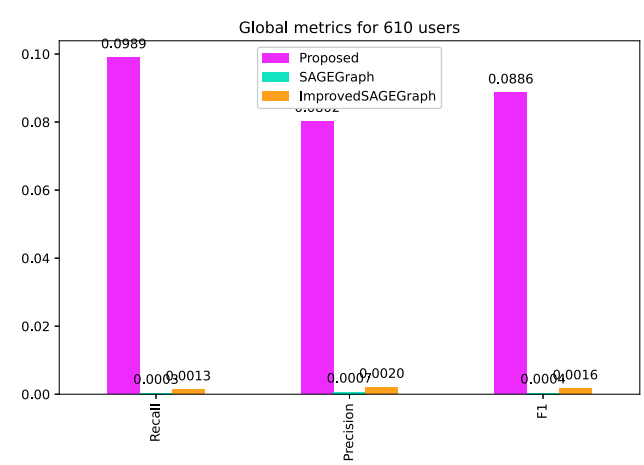

Fig. 11. The global metrics for 610 users.

Adopting our improved technique, offers several advantages over these alternatives in the case of recommender systems:

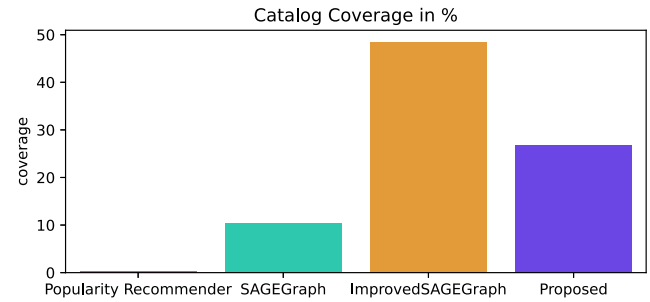

Fig. 12. Different models' coverage.

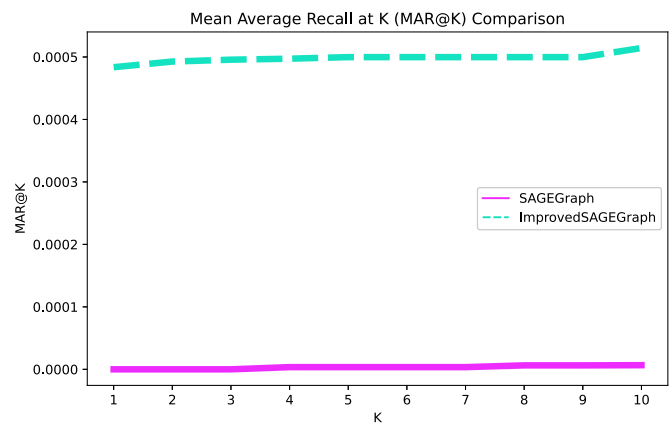

Fig. 13. Mean Average Recall at K.

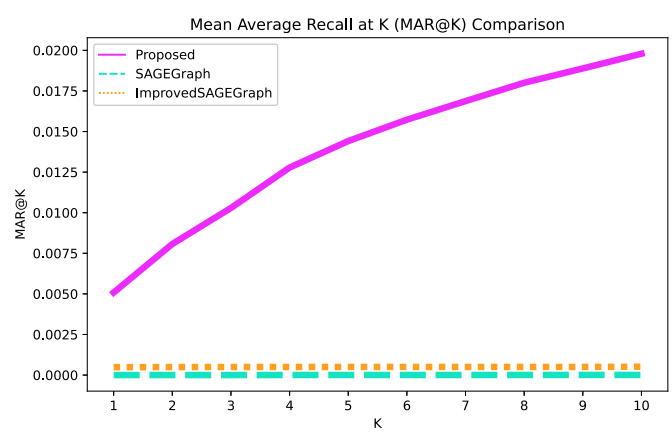

Fig. 14. Average mean recall at K for various models.

1. Heterogeneous Graph Modeling: The use of a heterogeneous graph-based model allows for the incorporation of various types of data and relationships, enabling a more comprehensive representation of user–item interactions and potentially capturing more complex patterns.
2. Neighborhood Aggregation with Activation Functions: The introduction of the Hyperbolic Tangent (Tanh) function followed by Rectified Linear Units (ReLU) as activation functions in the neighborhood aggregation phase can enhance the model's ability to capture non-linear relationships and complex interactions between nodes in the graph.
3. Cosine similarity : By utilizing cosine similarity to identify the top-N neighbors for users, recommender systems can leverage these advantages to improve the accuracy and relevance of recommendations. The measure takes into account the relative

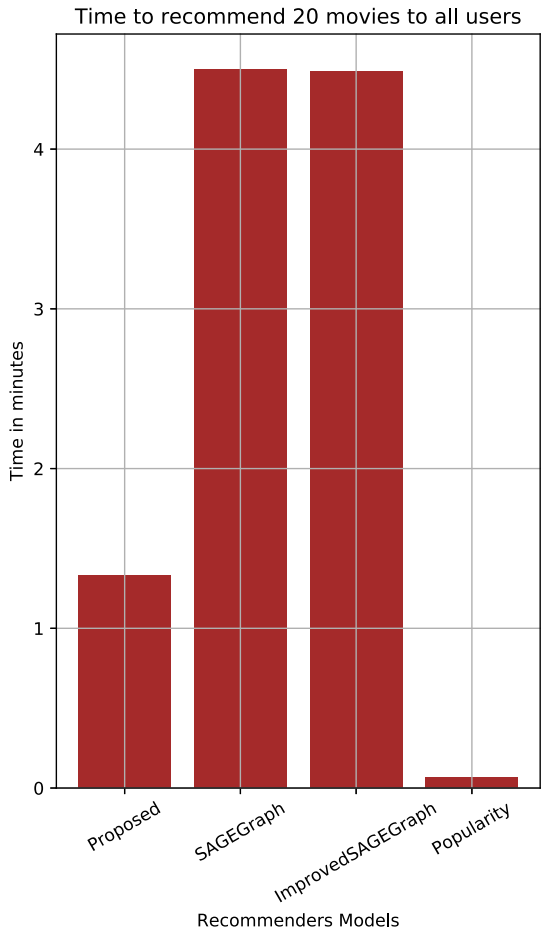

Fig. 15. Time to recommend Top-20 movies to all users.

importance and direction of user preferences, handles high-dimensional and sparse data effectively, and provides robustness to noise and irrelevant attributes, resulting in more personalized and meaningful recommendations for users.

4. Spectral Clustering: The utilization of Spectral Clustering can help identify groups or clusters within the graph, which can provide valuable insights into user preferences and item similarities. This can lead to more accurate recommendations by considering the local structure of the graph.
5. Loss function: RMSE prioritizes accurate predictions by penalizing large errors, providing an intuitive measure of prediction accuracy. It is compatible with optimization algorithms, ensuring efficient model training. By focusing on minimizing overall prediction error, RMSE emphasizes the importance of accurate recommendations in the context of the improved GraphSAGE approach.
6. Link Prediction for Suggestions: The incorporation of link prediction techniques can aid in generating suggestions by predicting missing or potential connections between users and items. This can help identify relevant recommendations even in scenarios with sparse or incomplete data.

Indeed, it is important to acknowledge the limitations of our suggested work. Two notable limitations include the reliance on a single dataset and the assumption of knowing the user's desired film year.

Using only one dataset can introduce potential biases and limitations to the findings. Dataset characteristics, such as its size, diversity, and specific user preferences, can significantly impact the performance and generalizability of the proposed approach. Therefore, it is crucial to validate the effectiveness of the method on multiple datasets to ensure its robustness and applicability across different contexts.

Additionally, assuming that the user's desired film year is known may not always hold true in practical scenarios. In reality, users may not have a specific preference or may be open to exploring films from various time periods. Failing to account for this variability can limit the accuracy and relevance of the recommendations provided. Future research could explore techniques to handle such uncertainty or investigate alternative methods that do not rely heavily on the user's desired film year.

By acknowledging these limitations, researchers can be aware of the boundaries and potential biases of the proposed approach. Addressing these limitations in future work, such as using multiple datasets and developing more flexible recommendation models, will lead to more comprehensive and reliable recommender systems.

5. Conclusion and future work

In this study, we suggested a method for generating movie recommendations using a heterogeneous graph-based model. Data preprocessing, building the heterogeneous graph, creating an Improved SAGEGraph model, employing Spectral Clustering, and producing suggestions using link prediction were all phases in our suggested methodology. Our early findings show that our strategy has the potential to improve the precision and diversity of movie recommendations.

Our method is noteworthy for a number of reasons. It starts by using a graph-based model, which can capture the complex connections between users and movies and produce suggestions that are more precise and varied. The model is trained on important and instructive features due to the use of feature engineering techniques.

In future research, there are several avenues to explore in order to overcome limitations and expand the scope of the recommended strategy.

One direction could involve investigating the applicability of our method in different contexts such as book or music recommendations. By testing the effectiveness of our approach across various domains, we can assess its generalizability and potential for wider applications.

Additionally, considering the temporal component of user preferences could be beneficial for enhancing the recommendation process. Exploring the usage of temporal models that incorporate time-related information, such as the sequence of user interactions or the evolution of preferences over time, could lead to more accurate and personalized recommendations.

Another area of interest lies in exploring more complex deep learning approaches (Afan et al., 2021). By delving into advanced deep learning models, we can potentially uncover novel techniques to further enhance the performance of recommendation systems. Moreover, incorporating prior knowledge or external data sources into the deep learning framework could help improve recommendation quality and cater to specific user preferences or domain-specific characteristics.

Efforts to accelerate the deep learning training phase can also be valuable. Researching techniques to reduce training time while maintaining or even improving model performance can lead to more efficient and scalable recommendation systems. This can involve exploring parallel computing, model compression, or other optimization strategies to streamline the training process.

By focusing on these aspects in future research, we can address limitations, expand the scope of our recommendation strategy, and contribute to the advancement of recommender systems in terms of performance, personalization, and efficiency.

CRediT authorship contribution statement

Yassine Afoudi: Data curation, Writing – original draft, Software, Validation. **Mohamed Lazaar:** Conceptualization, Methodology, Supervision. **Safae Hmaidi:** Visualization, Investigation, Review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

We would like to express our gratitude to all the individuals and organizations who have contributed to this work also we thank the reviewers for their valuable comments and feedback, which helped us improve the quality of this paper.

We would also like to thank the creators and maintainers of the dataset used in this work for making their data available to the research community.

References

Afan, Haitham Abdulmohsin, Osman, Ahmedbahaaaldin Ibrahem Ahmed, Essam, Yusuf, Ahmed, Ali Najah, Huang, Yuk Feng, Kisi, Ozgur, Sherif, Mohsen, Sefelnasr, Ahmed, wing Chau, Kwok, El-Shafie, Ahmed, 2021. Modeling the fluctuations of groundwater level by employing ensemble deep learning techniques. Eng. Appl. Comput. Fluid Mech. 15 (1), 1420–1439.

Afoudi, Yassine, Lazaar, Mohamed, Achhab, Mohamed Al, 2019a. Collaborative filtering recommender system. In: Advanced Intelligent Systems for Sustainable Development (AI2SD'2018) Volume 5: Advanced Intelligent Systems for Computing Sciences. Springer, pp. 332–345.

Afoudi, Yassine, Lazaar, Mohamed, Al Achhab, Mohamed, 2019b. Impact of feature selection on content-based recommendation system. In: 2019 International Conference on Wireless Technologies, Embedded and Intelligent Systems (WITS). pp. 1–6.

Aggarwal, Charu C, Aggarwal, Charu C, 2016. An introduction to recommender systems. In: Recommender Systems: the Textbook. pp. 1–28.

Al-Bakri, Nadia F, Hashim, Soukaena Hassan, 2018. Reducing data sparsity in recommender systems. Al-Nahrain J. Sci. 21 (2), 138–147.

Banan, Ashkan, Nasiri, Amin, Taheri-Garavand, Amin, 2020. Deep learning-based appearance features extraction for automated carp species identification. Aquac. Eng. 89, 102053.

Bathla, Gourav, Singh, Pardeep, Kumar, Sunil, Verma, Madhushi, Garg, Deepak, Kotecha, Ketan, 2021. Recop: Fine-grained opinions and sentiments-based recommender system for industry 5.0. Soft Computing. pp. 1–10.

Ben Schafer, J., Frankowski, Dan, Herlocker, Jon, Sen, Shilad, 2007. Collaborative Filtering Recommender Systems. Springer-Verlag, Berlin, Heidelberg, pp. 291–324.

Bobadilla, Jesús, Ortega, Fernando, Hernando, Antonio, de Rivera, Guillermo Glez, 2013. A similarity metric designed to speed up, using hardware, the recommender systems k-nearest neighbors algorithm. Knowl.-Based Syst. 51, 27–34.

Burke, Robin, 2002. Hybrid recommender systems: Survey and experiments. User Model. User-Adapt. Interact. 12, 331–370.

Cheng, Heng-Tze, Koc, Levent, Harmsen, Jeremiah, Shaked, Tal, Chandra, Tushar, Aradhye, Hrishi, Anderson, Glen, Corrado, Greg, Chai, Wei, Ispir, Mustafa, et al., 2016. Wide & deep learning for recommender systems. In: Proceedings of the 1st Workshop on Deep Learning for Recommender Systems. pp. 7–10.

Fan, Wenqi, Ma, Yao, Li, Qing, He, Yuan, Zhao, Eric, Tang, Jiliang, Yin, Dawei, 2019. Graph neural networks for social recommendation. In: The World Wide Web Conference. pp. 417–426.

Fan, Yajun, Xu, Kangkang, Wu, Hui, Zheng, Ying, Tao, Bo, 2020. Spatiotemporal modeling for nonlinear distributed thermal processes based on KL decomposition, MLP and LSTM network. IEEE Access 8, 25111–25121.

Gao, Chen, Wang, Xiang, He, Xiangnan, Li, Yong, 2022. Graph neural networks for recommender system. In: Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining. pp. 1623–1625.

García-Crespo, Ángel, López-Cuadrado, José Luis, Colomo-Palacios, Ricardo, González-Carrasco, Israel, Ruiz-Mezcua, Belén, 2011. Sem-fit: A semantic based expert system to provide recommendations in the tourism domain. Expert Syst. Appl. 38 (10), 13310–13319.

Guo, Huifeng, Tang, Ruiming, Ye, Yunming, Li, Zhenguo, He, Xiuqiang, 2017. Deepfm: A factorization-machine based neural network for CTR prediction. arXiv preprint arXiv:1703.04247.

Hajiramezanali, Ehsan, Hasanzadeh, Arman, Narayanan, Krishna, Duffield, Nick, Zhou, Mingyuan, Qian, Xiaoning, 2019. Variational graph recurrent neural networks. In: Advances in Neural Information Processing Systems, Vol. 32.

Hamilton, Will, Ying, Zhitao, Leskovec, Jure, 2017. Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems, Vol. 30.

Hanani, Uri, Shapira, Bracha, Shoval, Peretz, 2001. Information filtering: Overview of issues, research and systems. User Model. User-Adapt. Interact. 11, 203–259.

He, Xiangnan, Deng, Kuan, Wang, Xiang, Li, Yan, Zhang, Yongdong, Wang, Meng, 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 639–648.

He, Xiangnan, Liao, Lizi, Zhang, Hanwang, Nie, Liqiang, Hu, Xia, Chua, Tat-Seng, 2017. Neural collaborative filtering. In: Proceedings of the 26th International Conference on World Wide Web. pp. 173–182.

Hogan, Aidan, Blomqvist, Eva, Cochez, Michael, d'Amato, Claudia, de Melo, Gerard, Gutierrez, Claudio, Kirrane, Sabrina, Gayo, José Emilio Labra, Navigli, Roberto, Neumaier, Sebastian, et al., 2021. Knowledge graphs. ACM Comput. Surv. 54 (4), 1–37.

Huang, Qirui, Ding, Huan, Razmjooy, Navid, 2023. Optimal deep learning neural network using issa for diagnosing the oral cancer. Biomed. Signal Process. Control 84, 104749.

Idrissi, Nouhaila, Zellou, Ahmed, 2020. A systematic literature review of sparsity issues in recommender systems. Soc. Netw. Anal. Min. 10, 1–23.

Jain, Sarika, Grover, Anjali, Thakur, Praveen Singh, Choudhary, Sourabh Kumar, 2015. Trends, problems and solutions of recommender system. In: International Conference on Computing, Communication & Automation. IEEE, pp. 955–958.

Jin, Yucheng, Tintarev, Nava, Htun, Nyi Nyi, Verbert, Katrien, 2020. Effects of personal characteristics in control-oriented user interfaces for music recommender systems. User Model. User-Adapt. Interact. 30, 199–249.

Kipf, Thomas N, Welling, Max, 2016. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

Koren, Yehuda, Bell, Robert, Volinsky, Chris, 2009. Matrix factorization techniques for recommender systems. Computer 42 (8), 30–37.

Lika, Blerina, Kolomvatsos, Kostas, Hadjiefthymiades, Stathes, 2014. Facing the cold start problem in recommender systems. Expert Syst. Appl. 41 (4), 2065–2073.

Lops, Pasquale, Gemmis, Marco De, Semeraro, Giovanni, 2011. Content-based recommender systems: State of the art and trends. In: Recommender Systems Handbook. pp. 73–105.

Lops, Pasquale, Jannach, Dietmar, Musto, Cataldo, Bogers, Toine, Koolen, Marijn, 2019. Trends in content-based recommendation: Preface to the special issue on recommender systems based on rich item descriptions. User Model. User-Adapt. Interact. 29, 239–249.

Luxburg, Ulrike Von, 2007. A tutorial on spectral clustering. Stat. Comput. 17, 395–416.

Myers, Seth A, Sharma, Aneesh, Gupta, Pankaj, Lin, Jimmy, 2014. Information network or social network? the structure of the twitter follow graph. In: Proceedings of the 23rd International Conference on World Wide Web. pp. 493–498.

Ohsawa, Yukio, Benson, Nels E, Yachida, Masahiko, 1998. Keygraph: Automatic indexing by co-occurrence graph based on building construction metaphor. In: Proceedings IEEE International Forum on Research and Technology Advances in Digital Libraries-ADL'98. IEEE, pp. 12–18.

Porcel, Carlos, Moreno, Juan Manuel, Herrera-Viedma, Enrique, 2009. A multi-disciplinar recommender system to advice research resources in university digital libraries. Expert Syst. Appl. 36 (10), 12520–12528.

Ramzan, Bushra, Bajwa, Imran Sarwar, Jamil, Noreen, Amin, Riaz Ul, Ramzan, Shabana, Mirza, Farhan, Sarwar, Nadeem, 2019. An intelligent data analysis for recommendation systems using machine learning. Sci. Program. 2019, 1–20.

Razmjooy, Navid, Estrela, Vania V, Loschi, Hermes J, 2019. A study on metaheuristic-based neural networks for image segmentation purposes. In: Data Science. CRC Press, pp. 25–49.

Roy, Deepjyoti, Dutta, Mala, 2022. A systematic review and research perspective on recommender systems. J. Big Data 9 (1), 59.

Shinoda, Masahiro Moritaand Yoichi, 2012. Information filtering based on user behavior analysis and best match text retrieval. In: Proceedings of the Seventeenth Annual International ACM-SIGIR Conference on Research and Development in Information Retrieval. pp. 272–281.

Song, Jiagang, Song, Jiayu, Yuan, Xinpan, He, Xiao, Zhu, Xinghui, 2022. Graph representation-based deep multi-view semantic similarity learning model for recommendation. Future Internet 14 (2), 32.

Veličković, Petar, Cucurull, Guillem, Casanova, Arantxa, Romero, Adriana, Lio, Pietro, Bengio, Yoshua, 2017. Graph attention networks. arXiv preprint arXiv:1710.10903.

Wang, S., Cao, L., Wang, Y., Sheng, QZ., Orgun, MA., Lian, D., 2019a. A survey on session-based recommender systems. arXiv preprint arXiv:1902.04864.

Wang, Wen-chuan, Du, Yu-jin, Chau, Kwok-wing, Xu, Dong-mei, Liu, Chang-jun, Ma, Qiang, 2021. An ensemble hybrid forecasting model for annual runoff based on sample entropy, secondary decomposition, and long short-term memory neural network. Water Resour. Manag. 35, 4695–4726.

Wang, Xiang, He, Xiangnan, Cao, Yixin, Liu, Meng, Chua, Tat-Seng, 2019b. KGAT: Knowledge graph attention network for recommendation. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 950–958.

Wang, Xiang, He, Xiangnan, Wang, Meng, Feng, Fuli, Chua, Tat-Seng, 2019c. Neural graph collaborative filtering. In: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 165–174.

Wang, Hao, Wang, Naiyan, Yeung, Dit-Yan, 2015a. Collaborative deep learning for recommender systems. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1235–1244.

Wang, Hao, Wang, Naiyan, Yeung, Dit-Yan, 2015b. Collaborative deep learning for recommender systems. In: KDD '15, Association for Computing Machinery, New York, NY, USA, pp. 1235–1244.

Wang, Shu-Lin, Wu, Chun-Yi, 2011. Application of context-aware and personalized recommendation to implement an adaptive ubiquitous learning system. Expert Syst. Appl. 38 (9), 10831–10838.

Wibisono, Chrisna Haryo, Purwanti, Endah, Effendy, Faried, 2023. A systematic literature review of movie recommender systems for movie streaming service. In: AIP Conference Proceedings. Vol. 2554. AIP Publishing LLC, 040005.

Wu, Zonghan, Pan, Shirui, Chen, Fengwen, Long, Guodong, Zhang, Chengqi, Philip, S Yu, 2020. A comprehensive survey on graph neural networks. IEEE Trans. Neural Netw. Learn. Syst. 32 (1), 4–24.

Wu, Felix, Souza, Amauri, Zhang, Tianyi, Fifty, Christopher, Yu, Tao, Weinberger, Kilian, 2019. Simplifying graph convolutional networks. In: International Conference on Machine Learning. PMLR, pp. 6861–6871.

Wu, Shiwen, Sun, Fei, Zhang, Wentao, Xie, Xu, Cui, Bin, 2022. Graph neural networks in recommender systems: a survey. ACM Comput. Surv. 55 (5), 1–37.

Ying, Rex, He, Ruining, Chen, Kaifeng, Eksombatchai, Pong, Hamilton, William L, Leskovec, Jure, 2018. Graph convolutional neural networks for web-scale recommender systems. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 974–983.

Yu, Dongmin, Wang, Yong, Liu, Huanan, Jermsittiparsert, Kittisak, Razmjooy, Navid, 2019. System identification of pem fuel cells using an improved elman neural network and a new hybrid optimization algorithm. Energy Rep. 5, 1365–1374.

Zhang, Yin, Abbas, Haider, Sun, Yi, 2019a. Smart e-commerce integration with recommender systems. Electr. Mark. 29, 219–220.

Zhang, Shuai, Yao, Lina, Sun, Aixin, Tay, Yi, 2019b. Deep learning based recommender system: A survey and new perspectives. ACM Comput. Surv. 52 (1), 1–38.

Zhang, Jie, Zhao, Yihui, Shone, Fergus, Li, Zhenhong, Frangi, Alejandro F, Xie, Sheng Quan, Zhang, Zhi-Qiang, 2022. Physics-informed deep learning for musculoskeletal modelling: Predicting muscle forces and joint kinematics from surface emg. IEEE Trans. Neural Syst. Rehabil. Eng..

Zhou, Jie, Cui, Ganqu, Hu, Shengding, Zhang, Zhengyan, Yang, Cheng, Liu, Zhiyuan, Wang, Lifeng, Li, Changcheng, Sun, Maosong, 2020. Graph neural networks: A review of methods and applications. AI Open 1, 57–81.